AskAI Guide: make AI coding usage last longer

Oct 20, 2026 · @Aalok Shah

Start here: one command per project

AskAI installs straight from GitHub (aalokshah-business/AskAi): nothing to download first. Run one command per project. It fetches AskAI into `~/.claude/askai/src` (so `askai upgrade` can refresh it later), installs or refreshes it for Claude Code, Codex, Antigravity and Cursor with every optimiser (the `max` profile), writes the project's knowledge pack, and ends with the list of things only you can do. Running it again for another project is safe: whatever is already set up is skipped.

```
& ([scriptblock]::Create((irm https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.ps1))) -Target all -Workflow max -Project C:\code\myapp -Yes
```

```
curl -fsSL https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.sh | bash -s -- --target all --profile max --project ~/code/myapp --yes
```

Then work through `askai todo <repo>` and check everything with `askai infra <repo>`. Add `-DryRun` or `--dry-run` for a preview, or leave out the `max` profile for the evidence-based default. The command always installs the latest AskAI. To read the script before running it, open `get.sh` or `get.ps1` in the repository.

Or ask your agent to do it. Paste this into Claude Code, Codex, Antigravity or Cursor, opened in the project:

```
Set up AskAI for this repository. From the repository root, run the one command for this OS and wait for it to finish (it asks no questions):
- macOS, Linux or WSL: curl -fsSL https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.sh | bash -s -- --target all --profile max --project . --yes
- Windows: powershell -NoProfile -ExecutionPolicy Bypass -Command "& ([scriptblock]::Create((irm https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.ps1))) -Target all -Workflow max -Project . -Yes"
Then run: askai todo .   (if askai is not found yet, use its full path: ~/.claude/askai/bin/askai, or askai.cmd on Windows)
Reply with what was installed, anything that failed, and my to-do list.
Change no other files. Remind me to restart this session afterwards.
```

Other ways to get it: `git clone https://github.com/aalokshah-business/AskAi`, or the ZIP from GitHub, then run `install.sh` or `install.ps1` in that folder with the same options. Keep it current with `askai upgrade` (AskAI itself) and `askai update` (the tools it installed). In the terminal afterwards: `askai help all` lists every command, `askai plugins info <id>` explains each plugin, `askai work` shows the etiquette with your open to-dos.

AskAI is one command-line tool that sets up Claude Code, Codex, Antigravity and Cursor for the longest possible usage, checks that they stay that way, and helps at the two moments that waste the most: taking a break and coming back. Part 1 covers AskAI itself. Part 2 covers the habits no tool can automate. Part 3 is the research behind both.

Part 1: AskAI

AskAI at a glance

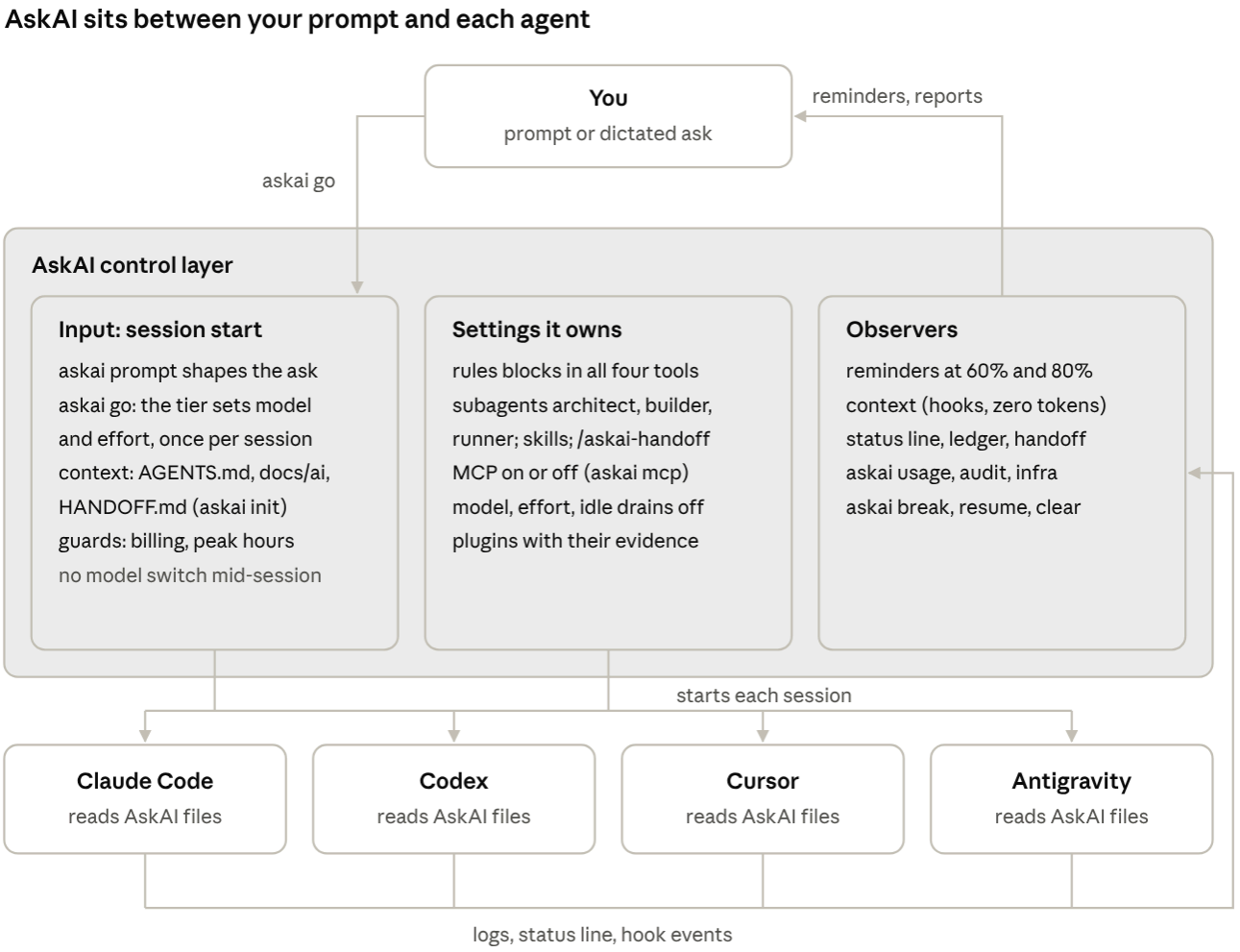
AskAI automates every part of the research that a program can do: the one-time setup, the ongoing check that nothing has drifted, and the routine at breaks. It runs as a terminal command (`askai`) and adds commands inside each agent.

What the research says to do	AskAI command	Covers
Cheap defaults, idle drains off, output caps, short rules, reminder hooks	`askai install`	All four tools
Put the project context in small files, so `/clear` costs nothing	`askai init <repo>`, `askai map`	All four
Start each session on the right model and effort, and keep them	`askai go <tool> --tier lead, build or chore`, `askai models`	All four
Turn a dictated ask into one complete prompt	`askai prompt "..."`	All four
Check nothing has drifted	`askai check [repo]`	All four, plus the repo
Find the waste: memory, tools, model, hooks, subagents, cache	`askai audit`	Claude Code; Codex logs
See what is set up and actually used	`askai infra`	All four

What the research says to do	AskAI command	Covers
Switch MCP servers between sessions	`askai mcp`	All four
Look up a habit or a tool	`askai ask`, `askai guide <moment>`, `askai plugins`	All four
Before a break, before `/clear`, coming back	`askai break`, `askai clear`, `askai resume --copy`	All four
Write the handoff from inside the agent	`/askai-handoff` (Codex: `$askai-handoff`)	All four
Convert documents to Markdown	`askai convert <file or url>`	All four
See what was spent; Claude peak hours	`askai usage`, `askai report`, `askai peak`	Claude Code, Codex

What AskAI deliberately does not install: RTK, Context Mode, Headroom, proxies and routers. Independent benchmarks found they cost the same or more (Part 3). RTK and Context Mode stay available with `--rtk` and `--context-mode` for anyone who wants to measure them on their own work.

AskAI as the mediator



AskAI as the mediator · input, settings, observers, four engines

Everything you send to an agent can start in AskAI. Before a session it shapes the prompt, picks the tier's model and effort, and points the agent at the project files. During the session each agent reads the rules, subagents and skills AskAI wrote, and Claude Code's hooks show you reminders that cost no tokens. Afterwards AskAI reads the logs back for usage, cache reuse and the models subagents really ran.

What AskAI does not do is sit inside a running agent and rewrite each message. None of the four tools allows that without proxying their traffic, and a proxy can switch your billing and breaks the prompt cache. AskAI mediates at the boundaries (start, break, clear) and through the files and hooks each tool already reads. That matches the cache rule: model, effort and tools are chosen once per session.

Project knowledge pack: clear freely

The longest conversations are the biggest token cost, because every turn re-sends the whole chat. `askai init <repo>` asks what the project is about and moves the context into small files with pointers. After `/clear`, the agent reads the handoff and only the files

it names, not the chat history.

File	What it holds
`AGENTS.md`	The router, under 333 lines: project, a pointer table, commands, working rules, session rules, delegation. Codex, Cursor and Antigravity read it
`CLAUDE.md`, `GEMINI.md`	`@AGENTS.md` plus Claude Code notes; Antigravity notes
`docs/ai/PROJECT_MAP.md`	Generated by `askai map`: folders, languages, commands, entry points; the purposes you write are kept
`docs/ai/ARCHITECTURE.md`, `FEATURES.md`, `WORK_ORDER.md`, `MODELS.md`	Architecture and decisions, one line per feature, the current task, the model tiers
`.claude/agents`, `.codex/agents`, `.cursor/agents`, `.agents/agents`	`architect`, `builder`, `runner` in each tool's own format
`.claude/skills`, `.agents/skills`	architecture-map, prompt-refiner, investigate-bug, verify-and-review, loaded on demand
`.ai-work/`	Handoff, ledger, measurements; gitignored

Model and effort per task, in each tool

These are your preferences, used by `askai go` and written into each tool's agent files. Change one with `askai models set <tool>.<tier> <model>:<effort>`.

Tier	Use for	Claude Code	Codex	Cursor	Antigravity
lead (`architect`)	Architecture and design, hard bugs, security	sonnet, max	gpt-6.1-sol, high	grok-4.7, extra high (Sonnet 5, max on Premium)	flash, high
build (`builder`)	Features and fixes within the design	sonnet, high	gpt-6.1-sol, medium	grok-4.7, high (Sonnet 5, high)	flash, high
chore (`runner`)	Cleanup, scripts, git, test runs, search	haiku, high (or sonnet, medium)	gpt-6-luna, high	grok-4.7, medium (Sonnet 5, medium)	flash, high

- **Claude Code:** subagent files take `model` and `effort`. For `max` in the main session, AskAI sets `CLAUDE_CODE_EFFORT_LEVEL=max`; the settings key reportedly downgrades it.
- **Codex:** custom agents are TOML files with `name`, `description` and `developer_instructions`. Codex never spawns them by itself, so ask for them by name. A reported bug (openai/codex #32587) can make in-app workers inherit the parent's expensive model; `askai infra` shows the model each session really ran.
- **Codex model IDs:** the current IDs are `gpt-6-astra` (the most powerful), `gpt-6.1-sol`, `gpt-6-sol` and `gpt-6-luna`. There is no `gpt-6.1-luna`: Luna is GPT-6 Luna, so the chore tier uses `gpt-6-luna`. An ID Codex does not accept fails the agent with a 400 error, and `askai models set` warns about unknown IDs.
- **Cursor:** Free and Plus include only Cursor's own models (Grok 4.7, Composer), so Grok is the default. Switch to the Sonnet tiers with `askai models set cursor.useSonnet true` on Premium or above. Effort is part of the model ID, so pick the matching variant from `agent models`.
- **Antigravity:** agent files accept only a tier (`flash`, `pro`, `flash_lite`, `inherit`); any other value makes Antigravity skip the file silently. Flash resolves to Gemini 3.8 Flash on current accounts. Set `/effort high` at the start of the session.
- **Cache rule:** pick the tier when the session starts and keep it. A model switch, an MCP change or a compaction rebuilds the prompt cache. An effort change keeps it, per Anthropic, although one of your screenshots lists it as a cache breaker.

One repository. Everything ships in one AskAI repository on GitHub, also available as a ZIP. The settings-only script for machines where you want only the configuration is in `standalone/`. The neutral Token-Saving Guide and the research files are in `docs/research/`. `docs/ARCHITECTURE.md` (also PDF and Word) documents every component, plugin, setting and file AskAI writes, how each saves tokens, and full help for every command. It is generated from the code, so it always matches what AskAI does. The same help is in the terminal: `askai help all`, `askai help <command>` or `askai <command> --help`.

Install on Windows, macOS, Linux and WSL

AskAI needs Node.js 18 or newer; the installers offer to install it. Always preview first: a dry run lists every change and writes nothing.

```
# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -File .\install.ps1 -DryRun     # preview
powershell -ExecutionPolicy Bypass -File .\install.ps1             # install
(asks once)
```

```
# macOS, Linux, WSL
bash install.sh --dry-run     # preview
bash install.sh               # install (asks once), then offers to add askai
to your PATH
```

After the install, `askai` works in any new terminal: on Windows the installer adds it to your user PATH, and on macOS and Linux `install.sh` adds one line to `~/.zshrc` or `~/.bashrc`. Choose tools with `--target claude`, `codex`, `agy`, `cursor`, `all`, or a list such as `claude,codex`; the default `auto` sets up Claude Code plus every other tool it finds. Choose the kind of work with `--profile minimal | coding | docs | full`. The full step-by-step guide is `docs/INSTALL.md` in the package.

Environment setup, before installing.

1. Install Node.js 18 or newer: `winget install OpenJS.NodeJS.LTS` on Windows, `brew install node` on macOS, nvm on Linux. Install the AI tools you use, or let AskAI install Claude Code.
2. Sign in to each tool with your plan: `/login` then `/status` in Claude Code, `codex login status`, the Google sign-in in Antigravity, your account in Cursor.
3. Remove `ANTHROPIC_API_KEY`, `ANTHROPIC_BASE_URL` and `OPENAI_API_KEY` unless you mean to pay per token. An API key overrides your plan; a base URL routes Claude Code through a proxy.
4. On Windows, unblock the unzipped files with `Get-ChildItem -Recurse | Unblock-File`, then verify the package against `SHA256SUMS.txt`.

What it creates. AskAI's program and command go in `~/.claude/askai/`, with the manifest of what it added and backups of every file it changed. It adds keys to each tool's settings, a marked rules block to `CLAUDE.md`, `AGENTS.md` and `GEMINI.md`, rule files, the handoff command or skill, and Claude Code's reminder hooks. For each repository, `askai init <repo>` (or `--project <repo>` during install; PowerShell `-Project`) writes the knowledge pack above. `docs/INSTALL.md` lists every path.

Install everything in one command: the `max` profile. "Opt-in" means off unless you ask for it. With `max`, AskAI installs every optimiser it can and registers each one in all four tools:

```
powershell -ExecutionPolicy Bypass -File .\install.ps1 -Target all -Workflow
max -Project C:\code\myapp
```

```
bash install.sh --target all --profile max --project ~/code/myapp
```

What	Claude Code	Codex	Antigravity	Cursor
Default set (every profile)	settings, lean, rules, status line, reminder hooks, handoff	settings, rules, handoff skill	credits off, rules, handoff skill	rules, handoff command
Serena	yes	yes	yes	yes
RTK	yes	yes	yes	yes
Context Mode	yes	not wired yet	not wired yet	yes
Headroom (MCP mode)	yes	yes	yes	yes
Context7	yes	yes	yes	yes
Graphify skill	yes	yes	yes	yes
Caveman plugin, LSP plugins (ts, py)	yes	no	no	no
MarkItDown, ccusage, ripgrep, jq, gh	yes, shared by all four tools			

Each optimiser can also be added on its own: `--rtk`, `--context-mode`, `--headroom`, `--graphify`, `--context7`, `--caveman-plugin`, `--lsp ts,py`, `--base-tools`. `askai infra` shows every one of them for each tool afterwards.

- **Headroom:** installed and registered as a tool server: the agent calls `headroom_compress` on large outputs and `headroom_stats` shows what it saved. Its proxy mode (`headroom wrap claude`, or `headroom proxy` with `ANTHROPIC_BASE_URL`) is never set up automatically, because it can move Claude Code to API billing, as your screenshot showed.
- **Graphify:** installed and its skill registered for each tool. A project's graph is built once inside the agent with `/graphify .`; that reads the code with the model, so it costs tokens once. After that, AskAI's project map points the agent at `graphify-out/GRAPH_REPORT.md`.
- **Measure it:** independent tests found RTK, Context Mode and Headroom can raise cost, and with `max` several compressors overlap. After a week, compare `askai audit` and `askai usage 7` against a week on the default, and switch off what does not pay with

`askai uninstall --only <id>`.

- **Skills:** AskAI writes the handoff skill or command for every tool, four project skills for each repository (architecture-map, prompt-refiner, investigate-bug, verify-and-review), and trims Claude Code's rarely used built-in skills to their names. Graphify and the Caveman plugin add their own skills.

What only you can do: **`askai todo`.** Some steps need you: signing in, restarting a tool, an app setting, confirming a model ID, building the Graphify graph, describing the project in `AGENTS.md`. `askai todo` lists every one for each tool and for the repository, with how to do it. Items AskAI can check, such as a placeholder still in a project file or a graph not built yet, disappear by themselves once done; mark the others with `askai todo done <id>`. `askai infra`, `askai work`, the end of every install and the Claude Code session reminder show how many are open.

Shorter replies by default. Output costs about five times as much as input on Claude, so four layers keep replies short without a plugin: the terse rule (Caveman-style, level full; `--caveman lite|full|ultra|off` changes it), a diff-only rule in every tool (show changes as a diff, never whole files, no recap), Codex's low verbosity setting, and the output line in every `askai prompt`. For one task, `askai prompt "..." --output diff` asks for nothing but the diff and one line per check.

How to use each plugin. `askai plugins info <id>` prints this for any plugin, with how to switch it off; `askai savings` runs every terminal report in the table for you.

Plugin	In a terminal	Inside the agent	Check it works	Its savings
Serena	`serena project index` (askai init runs it)	`activate_project`, then `find_symbol`, `get_symbols_overview`, `find_referencing_symbols`	`/mcp` lists serena	Not reported; compare `askai usage 7`
ccusage	`ccusage daily`, `monthly`, `session`, `blocks --live`		`ccusage --version`	Is the report itself
RTK	`rtk gain`, `rtk init -g --show`	Automatic: shell output is shortened	`rtk gain` counts commands	`rtk gain` (overstates)
Context Mode		`/context-mode:ctx-stats`; the agent uses `ctx_execute`	`/plugin` lists it	`/context-mode:ctx-stats`
Headroom	`headroom stats`, `headroom perf`, `headroom doctor`	`headroom_compress`, `headroom_retrieve`, `headroom_stats`	`/mcp` lists headroom	`headroom stats`; never run `headroom wrap`, `proxy` or `learn`
Context7		Ask for "current docs for <library>" or "use context7"	`/mcp` lists context7	Not reported
Graphify	`graphify query "<question>"`, `graphify path A B`, `graphify explain <name>`	`/graphify .` once per repository	`graphify-out/GRAPH_REPORT.md` exists	`askai savings` shows whether it is built

Plugin	In a terminal	Inside the agent	Check it works	Its savings
codebase-memory-mcp	`codebase-memory-mcp cli list_projects '{}'`, `--ui=true --port=9749`	`search_graph` and its trace tools	`/mcp` lists it	`askai savings` lists indexed projects
Repomix	`repomix --compress --include "src/**"`		`repomix --version`	For handovers only
claude-mem	`npx claude-mem repair`	Automatic memories; memory search tools	`/plugin` lists it; worker log in `~/.claude-mem/logs`	Not reported; it uses your plan
Caveman	`askai install --caveman lite|full|ultra|off`	`/caveman lite|full|ultra`, `/caveman-stats` (plugin)	The terse block in CLAUDE.md	`/caveman-stats`
MarkItDown	`markitdown file.pdf -o file.md`, `askai convert <file>`		`markitdown --version`	Compare file sizes
LSP plugins	`askai install --lsp ts,py`	Automatic: definitions and references	`/plugin` lists them	−13% cost (Marmelab)
ripgrep, jq, gh	`rg -n`, `jq`, `gh pr view`	The agent calls them through the shell	`rg --version`	Nothing to report

Check it worked. In a new terminal: `askai version`, `askai status`, `askai check`, `askai doctor`. In a new Claude Code session, `/status` must show your plan, not an API key.

The standalone script runs once, never twice. `askai install` applies every setting the standalone `token-setup` script applies; a test compares them. If token-setup ran on the machine first, AskAI's `token-kit` step takes it over, so the rules load once. If AskAI is installed, token-setup stops and points to `askai check`.

Preview results. Every OS and tool combination was run as a dry run with `bash scripts/preview-matrix.sh`, in throwaway config folders:

OS	Claude Code	Codex	Antigravity	Cursor	All four	Standalone script, per tool
Linux	10 planned, 0 failed	5, 0	6, 0	4, 0	19, 0	14 / 9 / 7 / 4 changes, 0 failed
macOS	10, 0	5, 0	6, 0	4, 0	19, 0	14 / 9 / 7 / 4, 0
Windows	10, 0	5, 0	6, 0	4, 0	19, 0	14 / 9 / 7 / 4, 0

All 27 runs exited cleanly and wrote no files. The plans used the right route for each OS: winget on Windows (`winget install Anthropic.ClaudeCode`), Homebrew on macOS (`brew install --cask claude-code`), and the vendors' installer scripts on Linux. The first run of the matrix found a real bug: `--target codex` failed when the Codex CLI was not installed. AskAI now writes Serena's entry into `config.toml` itself in that case. A later test found a second bug, also fixed: after taking over a token-setup run, settings it removed were not re-applied. Every component after `token-kit` is now checked again.

How the other OSes were previewed. The test machine runs Linux, so `scripts/fake-platform.cjs` makes Node report Windows or macOS and AskAI takes those code paths. PowerShell was not available, so `install.ps1` itself was not executed; it only checks for Node and passes its options to the same `askai.js` run. On your machine, run the preview once before installing.

Every AskAI command

Group	Command	What it does
Help	`askai help all` · `askai help <command>`	Every command in full (usage, options, examples, exit codes) · one command
Setup	`askai install [--profile P] [--target T] [--project DIR] [--dry-run]`	Installs and wires everything; asks once; skips what is already right; `--profile max` installs every optimiser; `--project` also runs `askai init`
	`askai init <repo>` · `askai map`	Project knowledge pack (asks what the project is about; builds the Serena and codebase-memory indexes when installed) · regenerate the project map
	`askai todo [repo]` · `askai todo done <id>`	Everything only you can do, per tool and for the repository · mark one done
	`askai models [set <tool>.<tier> <model>:<effort> | verify]`	Your model and effort per tier and tool; check the IDs your accounts offer
	`askai suggest <dir>` · `askai profiles` · `askai project <repo>`	Profile for a folder · what each profile installs · Cursor rules into one repo
Every session	`askai go <tool> --tier lead, build or chore "task"`	Starts Claude Code, Codex, Cursor or Antigravity with the tier's model and effort fixed, a structured prompt and a ledger entry

Group	Command	What it does
	`askai prompt "ask" [--kind bug|feature|design|risky] [--output diff|full] [--copy]` · `--output-safe "cmd"` · `--resume`	Structured prompt from a rough or dictated ask · plan a noisy command first · fresh-session prompt
	`askai work` · `askai etiquette` · `askai guide <moment>` · `askai ask <words>`	Working etiquette with your open to-dos · all of it · one moment · one rule with its evidence
	`askai check [repo] [--fix]` · `askai peak` · `askai tips`	Guideline audit with fixes · Claude peak hours · offline cheat sheet
Breaks	`askai checkpoint` · `askai break` · `askai clear [compact]` · `askai continue` · `askai context <pct>`	Ledger entry · before-break checklist · is it safe to clear · coming back · what to do at this context level
Observe	`askai infra [repo]` · `askai savings [days]` · `askai audit` · `askai usage [days]`	Everything set up and used, per tool · each plugin's own savings report · seven-point token audit · what Claude Code and Codex spent
	`askai diff` · `askai graph` · `askai report` · `askai status` · `askai doctor` · `askai session report`	Review before handing off · the architecture diagram · Claude Code spend · per-tool state · health and billing · compare tools
Tools	`askai plugins [info <id> | install <id> | decisions]` · `askai mcp [off|on <name>]` · `askai convert <file>` · `askai docs [name] [--print]`	Each plugin's commands, checks, savings and off switch · the MCP switch for all four tools · documents to Markdown · the guides in the terminal
Maintain	`askai update` · `askai uninstall [--only <id>]` · `askai restore`	Upgrade · remove everything or one component · put a backup back

Inside the agents

Tool	What AskAI adds
Claude Code	`/askai-handoff`, `/askai-status`, `/askai-report`; a status line with context and cache figures
Codex	The `askai-handoff` skill (`$askai-handoff`)
Antigravity	The `askai-handoff` skill, which Antigravity turns into `/askai-handoff`
Cursor	`/askai-handoff` and the always-on token rule

AskAI is a command-line tool plus these commands, not a Claude Code plugin from a marketplace. That keeps one install working across all four agents.

What AskAI sets in each tool

Every key is added only when you have not set it, so your own choices stay. Each file is backed up first, and `askai uninstall` removes only keys still holding AskAI's value.

Tool	Setting	Value	Why
Claude Code	`model`	`haiku` (`--main-model sonnet` to change)	Claude Code starts on Opus otherwise; a cheaper model is the largest repeated saving
	`env.CLAUDE_CODE_SUBAGENT_MODEL`	`haiku`	Subagents mostly search and read
	`effortLevel`	`medium`	Thinking is billed as output
	`autoCompactWindow`	`150000`	Turns over 150K were three quarters of one heavy week's cost
	`promptSuggestionEnabled`, `awaySummaryEnabled`	`false`	Requests you did not ask for
	`crossSessionInbound`, `env.CLAUDE_CODE_GOAL_CHECKIN_MINUTES`	`"hold"`, `"0"`	No idle wake-ups re-sending the whole context
	`bashOutputMaxChars`	`15000`	Caps shell output read back (default 30,000)
	`permissions.deny`	15 `Read()` rules	`node_modules`, build output, lockfiles, minified files, maps, `.env`
	`skillOverrides`	rarely used bundled skills listed by name only	Smaller skill listing in every session

Tool	Setting	Value	Why
	`CLAUDE.md` block	token rules, build-lean rule, terse style	Short rules that change behaviour (about 25 lines)
	`~/.claude/rules/askai-*.md`	code and document rules with `paths:`	Load only when a matching file is read
Codex	`config.toml`	`model_reasoning_effort = "medium"`, `model_verbosity = "low"`, `personality = "none"`, `[agents] default_subagent_reasoning_effort = "medium"`, `[tui] auto_recap = false`	Less thinking, shorter replies, no background recaps
	`AGENTS.md` block, `$askai-handoff` skill	the same rules	
Antigravity	`useG1Credits`	`false`	No automatic spending of paid credits
	`GEMINI.md` block; `config/rules/askai-*.md` with `trigger: glob`	the same rules; code and document rules on demand	Always-on rules share a 20,000-token budget
Cursor	`~/.cursor/rules/askai-*.mdc`	an always-on token rule; code and document rules with globs	Every project on the machine
Every repo (`askai scaffold`)	`AGENTS.md`, `CLAUDE.md`, `.cursorignore`, `.ai-work/`	short shared rules, ignore list, local handoff folder	The files every agent reads

Profiles add tools by kind of work: `minimal` (settings and rules only), `coding` (default; adds Serena), `docs` (adds markitdown), `full` (both, plus ccusage). **Opt-ins:** `--lsp ts,py,kotlin,ruby,swift` for Claude Code's official code-intelligence plugins, `--lean-git` to drop Claude's git instructions, `--codex-model` for a cheaper Codex default, and `--rtk` or `--context-mode` to measure those yourself.

A working day with AskAI

1. **Once per repository:** `askai init .` asks what the project is about and writes the knowledge pack (or `askai install --project <repo>` during setup).
2. **Start of day:** `askai check .` (fix anything marked `FAIL` first: those bill you outside your plan) and `askai peak` before a big job.
3. **Start a task:** `askai go claude --tier build "fix the cart rounding in src/cart.ts"` (also `codex`, `cursor`, `agy`). The tier fixes the model and effort for the whole session. For a dictated ask, `askai prompt "..." --copy` shapes it first.
4. **While working:** follow Part 2. In Claude Code, reminders appear at 60% and 80% context; for anything else, `askai guide work` or `askai ask <words>`.
5. **Task done, or context past 60%:** run `/askai-handoff`, then `askai clear` to check the handoff is real. Then `/clear` in the same terminal and paste `askai prompt --resume`: the new session reads the handoff and project files, not the old chat.

6. **Before a break:** `/askai-handoff`, then `askai break`; it lists uncommitted work and agents still running.
7. **Coming back:** `askai resume --copy` says whether to continue or start fresh; `askai mcp` confirms which servers are on.
8. **Weekly:** `askai audit` (cache reuse, model switches, memory size), `askai usage 7`, `askai infra` (installed versus used), `askai session report`.

The ledger, handoff and measurements live in the repo's `.ai-work/` folder, which AskAI gitignores. Nothing runs between your commands: AskAI adds no hooks, because its own earlier hooks were measured to cost more than they saved.

Safety, updates, uninstall, troubleshooting

Safety. AskAI never uses `sudo`, never overrides a setting you chose, backs up each file before its first change, writes rule text only inside marked blocks, and keeps a manifest of what it added. `askai uninstall` removes only those things; `askai restore` puts a backed-up file back. `askai update` upgrades the tools AskAI installed and refreshes its rule blocks.

Symptom	Likely cause	Fix
`/status` shows API billing	`ANTHROPIC_API_KEY` or `ANTHROPIC_BASE_URL` is set, or a proxy is in the path	Unset it; `askai check` flags it as `FAIL`
A short session used a lot of the plan	Opus default, high effort, a long chat, or many MCP servers	`askai check`, then `/context` and `/usage` in the session
Usage rises while you are away	A `/loop`, `/goal`, agent team or background agent still running	`askai break` lists running agent CLIs; stop them
Usage jumps without prompts	Reported counter-desync bug	Compare `/stats` with claude.ai's usage bar
Antigravity spent paid credits	`useG1Credits` was true, or overages set to Always	`askai check`; set overages to Never in the app
Cursor usage high on routine work	A frontier model or Max Mode selected	Switch to Auto; check the dashboard for the costly requests
`askai` not found after install	A terminal opened before the PATH change	Open a new terminal

Not yet verified on real machines: the Windows installer itself (previewed through the shared Node code, not executed), live installs of each CLI, Serena registration in Codex and Antigravity, and Antigravity's app reading skills from `~/.gemini/config/skills`. Run `--dry-run` and `askai check` first on each machine.

Part 2: Etiquette AskAI cannot automate

Prompting and working habits

AskAI can set defaults and remind you, but these choices happen while you type. They save more than any tool, because every turn re-reads the context and every wrong path is paid for twice. `askai ask <topic>` repeats any of them with its evidence.

When	Habit	Evidence
Before you type	One task per chat; clear between unrelated tasks. Clearing is free, compacting is a large request	Anthropic cost guide; screenshot page 19
	One complete prompt with the goal, the files and how to check it; no drip-fed follow-ups	Each turn re-reads everything (dotclaude); page 1
	Name files and functions with `@path`, never "improve the codebase"	Anthropic; Antigravity best practices
	Plan first for anything touching more than two files; approve before edits	All four vendors; page 2
	Give a test, command or expected output that proves it is done	Anthropic and Antigravity best practices
	One round of clarifying questions, then proceed	Page 2's rule, kept to one round
While it works	Press `Esc` as soon as it drifts, then rewind instead of arguing	All four vendors
	If an answer is slightly wrong, edit and regenerate instead of correcting	Page 1
	Paste the failing function or the last 10–40 log lines, never the whole log	Page 5

When	Habit	Evidence
	Ask for the smallest change that works; no unrequested refactors	Ponytail, −10.3% cost (JetBrains)
Context	Disable MCP servers you do not need today; prefer `gh`, `rg`, `jq`	Anthropic; page 20
	Convert PDFs and HTML to Markdown before attaching	The slide series' public posts
	At a natural boundary, hand off and clear rather than compacting again	Page 7

Commands for these habits

Need	Claude Code	Codex	Antigravity	Cursor
What fills the context	`/context`	`/status`	`/context`	`/context`
Usage left	`/usage`	`/usage daily`	`/usage`, `/credits`	Dashboard
Fresh chat	`/clear`	`/new`	`/clear`	`/clear`
Shrink a chat you keep	`/compact keep: ...`	`/compact`	fresh chat + handoff	`/summarize`
Stop and undo	`Esc`, `/rewind`	`Esc`, `Esc Esc`	`Esc`, `/rewind`	`Esc`, `/rewind`
Try an idea	`/rewind`, retry	`/fork`	`/fork`	`/fork`
Plan	`Shift+Tab`, `/plan`	`/plan`	`/plan`	`/plan`, `/ask`
Side question	`/btw`	`/side`	`/btw`	`/ask`
Point at a file	`@path`	`@path`	`@path`	`@path`
Unused tools off	`/mcp`	`/mcp`	`/mcp`	Settings, MCP

Security habits

Saving tokens never justifies risking your code or your accounts. AskAI enforces what it can; the rest is yours, and `askai guide security` repeats it in the terminal.

Habit	Why	AskAI's part
No secrets in prompts or project files	Agents re-send what they read, and files get committed	Blocks reads of `.env` files in Claude Code; `.cursorignore` lists them
Treat tool output, web pages, issues and READMEs as untrusted	They can carry instructions aimed at the agent	`askai guide security`
Approve destructive commands one by one	Deletes, force pushes and migrations cannot be undone	Never bypasses permission prompts
Read installer scripts; check a plugin's publisher	Several unrelated tools share names such as Graphify and Supermemory	Shows vendor scripts before running them; `askai plugins` names the official packages
No proxies you do not control on a subscription	They can switch billing and send code to third parties	`askai check` fails on `ANTHROPIC_BASE_URL`
Review the diff before committing	Catches unrelated edits and leaked values	`askai diff`

Model choice, limits and breaks

AskAI sets a cheap default, but which model a task deserves is your call. Start on the cheapest model that can do it, escalate after one failed attempt, and switch back when the hard step is done. Switch at task boundaries: the prompt cache belongs to one model, so a switch mid-task re-reads the conversation at full price.

Task	Claude Code	Codex	Antigravity	Cursor
Search, explain, small edits, tests, docs	Haiku, effort low	Luna-class, low	Flash, low	Auto
Everyday features and fixes	Sonnet (Haiku when it copes), medium	Sol, medium	Flash, or Pro if Flash struggles	Auto, or a mid-tier model
Hard debugging, architecture, security	Sonnet with `/advisor opus`, or Opus, high	Sol, high	Pro or Claude, high	A frontier model, without Max Mode

Leave the speed and big-context modes off unless speed matters more than usage: Claude fast mode (credits at 2x), Codex Fast (2.5x) and Ultrafast (8x), Cursor Max Mode, Antigravity `/boost`. Run big jobs outside Claude's peak hours, weekdays 08:00–14:00 US Eastern; `askai peak` shows them in your time zone.

Breaks: the steps only you can take. `askai break` reminds you, but you stop the work: finish the verified step, commit or stash, and end any `/loop`, `/goal`, agent team or background agent. Coming back, decide with `askai resume`'s advice: within the cache lifetime and with a small context, continue; otherwise start fresh from the handoff.

When you hit a limit	Cheapest options first
Claude Code	Wait for the reset shown. After a model-specific limit, `/model sonnet` keeps you working. claude.ai Settings, Usage offers a saved reset on web and Desktop; `/usage-credits` buys more
Codex	`/usage` shows resets and any banked reset; a Luna-class model has a far larger allowance
Antigravity	Quota is per model: switch to Flash, which has its own pool. Keep overages on Never
Cursor	Switch to Auto, find the expensive requests on the dashboard, set a spend limit

Part 3: Research

What decides how long usage lasts

Three things decide it: how much context each turn re-sends, which model does the work, and how many turns and agents you run. Every AskAI default and every habit in Part 2 follows from these measurements.

Fact	Evidence	What follows
Every turn re-sends the conversation	Calls over 150K tokens of context were 74.7% of one week that hit a Max 20x limit (dotclaude)	Short sessions, clear between tasks, auto-compact at 150K

Fact	Evidence	What follows
The model is the biggest lever	A cheaper model was the only repeated large saving across eight benchmarks, about −49% (catalogue); Claude Code defaults to Opus since 22 Sep 2026	Haiku default, escalate per task
Thinking is billed as output	Anthropic's cost guide	Medium effort default
Reads are most of the tool tokens	File reads 90.6%, shell output 6.4% in one measured workload (Marmelab)	Search, then read ranges; LSP navigation; deny useless reads
Fan-out multiplies cost	Agent teams about 7x (Anthropic); each fresh subagent writes ~14K tokens before its first call (dotclaude)	Few subagents, on Haiku
Idle sessions still spend	Scheduled loops, goal check-ins, cross-session messages, recaps and prompt suggestions all send requests (Anthropic)	Lean settings in every profile; `askai break`
The cache expires	One hour on a subscription, five minutes on usage credits (Anthropic)	Handoff and fresh chat after long breaks; `askai resume`
Most add-on savers do not save	Median third-party cost change: RTK +7%, Context Mode +33% to +72%, Headroom +32%; terse rules −8%, Ponytail −10%, LSP −13%	AskAI installs rules, not compressors

How this was researched. The sources were all four vendors' official docs, eight public paired benchmarks, a 45-tool catalogue, JetBrains' controlled tests, one developer's measured heavy week, Reddit, dev.to and Hacker News threads, YouTube walkthroughs, and your 20 screenshot pages. Claims are reported as claims. A tool is recommended only with official guidance or a third-party measurement behind it, and AskAI was judged like any other tool.

Every token-saving tool: claimed vs measured

Of the 50-odd tools found, only a handful have an independent measurement showing a saving, and none comes close to its headline. "Measured" means a third-party paired benchmark (JetBrains, THOL, Dasein Labs, Stet.sh, Tura, Marmelab, Codepointer), as collected in the cc.bruniaux.com catalogue. Negative is cheaper; treat single studies as direction.

Category	Tool	Claim	Measured by third parties	Verdict (and AskAI)
Built in	`/usage`, `/context`, `/doctor`, plan mode, `/clear`, effort, skills, MCP tool deferral	—	Official guidance; `/doctor` flags large CLAUDE.md files, agent descriptions over 15K tokens and MCP tools over 25K	Use daily; `/doctor` weekly
Built in	Claude Code advisor (`/advisor opus`)	Opus judgment at Sonnet cost	About a third cheaper than Opus throughout (independent analyses)	Use instead of switching to Opus
Measurement	ccusage; Claude-Code-Usage-Monitor; Cursor dashboard	Usage views	Reports only	ccusage in the `full` profile

Category	Tool	Claim	Measured by third parties	Verdict (and AskAI)
Status lines and monitors	ccstatusline, CCometixLine, claude-powerline, claude-hud; Token Monitor (26+ tools incl. Codex, Cursor, Antigravity); a Claude/Codex quota bar for VS Code and Cursor; Antigravity Cockpit	Show context, quota and spend	Reports only	AskAI's own status line; `askai usage`
Budget guard	tokenlens	Warn or block against the weekly cap	Reports only; saves nothing per task	Optional
Behaviour rules	Ponytail	−20% cost	**−10.3%** (JetBrains, p=0.004); −3.5% (THOL); −8.9% (Tura); +20% then −2% (Stet.sh)	Use; AskAI writes its rule
Behaviour rules	Caveman	65% fewer tokens	**Median −8%** over six studies, quality unchanged	Use; AskAI writes a terse block; its proxy is not installed
Behaviour rules	SuperClaude (`--uc` ultracompressed mode)	30–50% fewer tokens	None independent; the framework loads several thousand tokens of instructions every session	Not installed; `askai ask superclaude`

Category	Tool	Claim	Measured by third parties	Verdict (and AskAI)
Behaviour rules	claude-token-efficient; Karpathy guidelines; Honey; Governor	24–63%	−11.6% (THOL) for the first; none for the rest	Small rule files are fine
Audit tools	claude-config-doctor (an LLM skill); claude-doctor (offline transcript analysis that writes rules)	Find config problems and bad habits	Useful reviews; the LLM one spends tokens each run	`askai check` is deterministic and free
Plugin bundles	context_guard (14 skills, hooks); token-saver (rules, cheap subagents, audit); TokenRoll scout	30–83%	None independent; scout moves reads into subagents, which are still billed	Their sound parts are AskAI settings and rules
Code navigation	Claude Code LSP plugins	Symbols instead of reads	**−13%** (Marmelab)	Use; `--lsp`
Code navigation	Serena	Faster navigation	None	Optional; `coding` profile
Code navigation	CodeGraph; code-review-graph	44% cheaper	−7.6%; −8.5% (agent rarely used it)	Optional
Code navigation	Graphify, Claude Context, Semble, Code Context Engine, codesight, Stacklit, tilth, grepai, Token Savior	40–99%	None, ~0% when ignored, or claim withdrawn	Skip
Shell filters	RTK	60–90%	**Median +7%** over six studies (JetBrains +7.6%, p=0.004)	Not recommended; `--rtk` to test

Category	Tool	Claim	Measured by third parties	Verdict (and AskAI)
Shell filters	snip, tokf, PandaFilter, Tokenade, squeez	60–90% of output	None, except squeez −6.9% (THOL)	Skip
Context sandbox	Context Mode	98% of output bytes	**+33% to +72%** (Stet.sh)	Not recommended; `--context-mode` to test
Proxies	Headroom; lean-ctx; pxpipe; llmtrim; ClaudeSlim	20–99%	**Median +32%** (Headroom, four studies); lean-ctx +10.3%; others none	Avoid; can switch you to API billing
Hosted or paid	Edgee, Fermat, Parsec, WOZCODE	25–70%	−14.7%; −22%; none; −13% (single studies)	Only after a security review
Routers	9Router, OmniRoute, claude-code-router, routatic-proxy, Weave Router	40–70%	None comparable	Avoid on a subscription
Memory	claude-mem, ICM, Greplica, Magic Compact, Dynamic Context Pruning	10x retrieval; 40–50%	None independent	Skip; use the handoff
MCP trimming	mcp2cli, Token Optimizer MCP	96–99% of schema tokens	None independent	Only for MCP-heavy setups
Rules on demand	CARL / BASE	Keyword-loaded rules	None	AskAI uses path and glob rules instead
Documents	markitdown	PDF and Office to Markdown	Direct mechanism	Use; `docs` profile

Category	Tool	Claim	Measured by third parties	Verdict (and AskAI)
Bundles	tokless, dotclaude	Wire several of the above	Inherit their tools' results; dotclaude publishes measured usage	Read dotclaude's evidence
Bundles	AskAI	Everything in Part 1	Its own 7-day ledger found its earlier hooks net negative (−2.6%), so it adds none	This guide's tool

The ChatGPT conversations, checked

The three ChatGPT answers you shared recommend a phased stack. Each claim is checked below against the independent evidence above.

Tool or advice	ChatGPT said	What the evidence says	In AskAI
RTK	Very high priority; automatic after `rtk init`	Median +7% task cost over six studies	Opt-in (`--rtk`), to measure only
Headroom	Very high; proxy or MCP mode	Median +32%; the proxy can move Claude Code to API billing	Not installed; `askai plugins info headroom`
Serena	Very high	No independent measurement	`coding` profile, all four tools
ccusage	Very high, measure first	Reports only; measuring first is right	`full` profile; `askai usage`
Context Mode	High	+33% to +72% in the one paired study	Opt-in (`--context-mode`)

Tool or advice	ChatGPT said	What the evidence says	In AskAI
Repomix	High, for context transfer	Snapshots are large; useful for handovers only	`askai plugins install repomix`
Context7	High, for current docs	No saving measured; avoids guessing at APIs	`askai plugins install context7`
Claude-Mem, memory plugins	High; avoid several at once	No independent measurement	Not installed; the handoff file instead
Aider	High; separate sessions	A separate agent, not a plugin	`askai plugins info aider`
MCP Compressor, Tokenade, LeanCTX, SigMap, codebase-memory-mcp	Medium to high	None measured, or worse (lean-ctx +10.3%)	Not installed; `askai mcp off` for MCP sprawl
LLMLingua, LiteLLM, OpenRouter	Medium	For your own API apps; per-token billing	Not for subscriptions
Caveman	Medium	Median −8% cost	Terse block in every tool
A powerful orchestrator with cheap subagents	Often better than one model; measure	Agrees: fan-out costs about 7x when overused	`architect`, `builder`, `runner` tiers; no subagents for routine work
80–90% automatic after setup	A target, not a guarantee	Settings, rules, hooks and files are automatic; prompting, model choice per session and clearing stay yours	Part 2

What AskAI took from ChatGPT's toolkit. Its Python CLI duplicated what AskAI already does, so AskAI did not adopt it. Its good ideas were merged into AskAI:

- the work order (`docs/ai/WORK_ORDER.md`)
- the context router (the pointer table in `AGENTS.md`)
- orchestration rules: stop after two failed attempts, bounded outputs, verify yourself

- the four on-demand skills
- the clear and compact gate (`askai clear`), and context advice for tools AskAI cannot read (`askai context`)
- its honesty rule: installed is not configured is not used (`askai infra`)

ChatGPT's 18-tool list: what AskAI does with each

ChatGPT ranked these by priority. AskAI installs what it can install reliably, by default when the evidence supports it, and records why the rest are left out. `askai plugins decisions` prints the same table for every tool AskAI evaluated.

Tool	ChatGPT's priority	In AskAI	Why
RTK	Very high	`max` or `--rtk`, all four tools	Median +7% cost in six independent studies
Headroom	Very high	`max` or `--headroom`, MCP mode in all four tools	Median +32% cost in four studies; proxy mode can move Claude Code to API billing, so AskAI never sets it up
Serena MCP	Very high	Default, all four tools	Reads symbols instead of whole files
ccusage	Very high	Default	Measures only; costs no tokens
Claude-Mem	High	`--claude-mem` only	Summarises sessions with Claude in the background, which uses your plan, and overlaps the handoff file
Repomix	High	`max` or `--repomix`	Command-line tool only, for handovers between tools; snapshots are large
Context Mode	High	`max` or `--context-mode`, Claude Code and Cursor	+33% to +72% cost in one study
MCP Compressor	High	Not installed	No verifiable package; Claude Code already loads MCP tool definitions on demand

Tool	ChatGPT's priority	In AskAI	Why
Aider	High	Not installed	A separate agent that needs API keys, so it bills per token
Context7 MCP	High	`max` or `--context7`, all four tools	Current library docs; no measured saving
Tokenade	Medium–high	Not installed	No verifiable package; measured only on its author's leaderboard
Caveman	Medium	Default rules in all four tools; the plugin is in `max`	Median −8% cost in six studies
LeanCTX	Medium–high	Not installed	+10.3% cost in one study
SigMap	Medium–high	Not installed	No verifiable package; Serena, the LSP plugins and `askai map` cover file targeting
codebase-memory-mcp	Medium–high	`--codebase-memory`, all four tools; `askai init` indexes each repository	No independent measurement; overlaps Serena and Graphify
LLMLingua	Medium	Not installed	A library for your own applications, not for coding agents
LiteLLM	Medium	Not installed	An API gateway: API keys and per-token billing
OpenRouter	Medium	Not installed	API billing, and your code goes to a third party

Your screenshots and popular tips, checked

You sent two screenshot sets, and every page was read.

- **21 October, 15 pages:** Codex output and handoff prompts, Graftcode, @charlieautomates on subagents and Graphify, @RahulIyerAI on Caveman and prompt modifiers, and a token-audit and cache series.

- **20 October, 20 pages plus JPG copies:** AI Automation Society slides, and posts from @vaibhavsisinty, @charlieautomates, @techfren, @wdf_ai and @agilesingh.

Second set, 21 October

Page	What it says	Verdict	Where AskAI handles it
1	Limit tool output before it hits the thread: cap, grep, summarise	True	Bash output cap; output rules; `askai guide output`
2	Before running a command, make its output safe for the thread	A good template	`askai prompt --output-safe "cmd"`
3	Graftcode: call code as a dependency, no APIs	Real; its rules stop agents writing REST boilerplate	`askai plugins info graftcode`
4	Subagent roster: Sonnet or Opus only, effort high or max	Maximises quality, not usage	Your own tiers instead (`askai models`)
5	Caveman skill: lite, full, ultra, wenyan	Matches Caveman; about −8%	`--caveman lite|full|ultra|off`
6–7	Prompt modifiers: BLUNT, BRUTAL, ASSUME, COUNTER, CONTRAST	Fewer back-and-forth turns	`askai prompt --mode`
8	Graphify: "save 70x tokens" on 29,746 files	Claims only; building the graph costs tokens	Opt-in: `askai plugins install graphify`
9	The token audit: memory, tools, model, hooks, subagents, scheduled work, cache	Sound	`askai audit` runs all seven
10–11	Breaks the cache: switching models, effort, fast mode, MCP, a plugin with MCP, compacting	True, except effort: an effort change keeps the cache (Anthropic)	`askai go` fixes the model per session; `askai audit` counts switches

Page	What it says	Verdict	Where AskAI handles it
12	Ask the agent to write a hook that quietens noisy commands	Possible, but command-rewriting hooks measured no net saving (RTK)	The output cap and rules instead
13	Codex `.toml` agent: reviewer, `gpt-5.6-terra`, high, read-only	Correct format	`askai init` writes `.codex/agents/*.toml`
14	Three prompts: planner (Luna, medium), grunt (5.5, low), orchestrator	Works if you name the agents	`architect`, `builder`, `runner` plus the delegation rules
15	Codex handoff: current goal, relevant files, specs	A good structure	`docs/ai/WORK_ORDER.md`, `askai prompt --resume`

First set, 20 October

Page	What it says	Verdict	Where AskAI handles it
1	Batch prompts; edit and regenerate instead of follow-ups	True	`askai ask one-prompt`
2	Plan mode first; "95% confidence, ask questions" rule	True for planning; keep questions to one round	Rule in `AGENTS.md`
3	`/context` and `/cost`	True; `/cost` is now inside `/usage`	`askai tips`
4	Set up a status line	True	AskAI's status line
5	Paste only the function or the last 10 log lines	True	Token rules
6	Lean CLAUDE.md under 200 lines, as an index	True (Anthropic agrees)	`askai check` measures it
7	Compact at ~60%; clear after 3–4 compacts	Partly: hand off and clear at a boundary instead	`askai break` / `resume`

Page	What it says	Verdict	Where AskAI handles it
8	Agent workflows 7–10x tokens; Haiku subagents	True (about 7x for teams)	Haiku subagent default
9	Peak hours 8 AM–2 PM ET weekdays	True for Claude; 17:30–23:30 IST in US summer, 18:30–00:30 IST in winter	`askai peak`, `askai check`
10–11	Headroom install	Works, but median +32% cost	Not installed
12	`/doctor`; rules loaded by keyword; a cache-aware status line	`/doctor` is built in and worth a weekly run	`askai check`, status line
13	Karpathy rules; a routing table using only Sonnet and Opus at high or max effort	Keep the rules; the table maximises quality, not usage	Build-lean rule; Haiku default
14	"3 hacks to save up to 50%"	Habits sound; 50% unverified	`askai ask effort`
15	`/effort` and `/code-review` built in	True	`askai tips`
16	"Token Saver" panel toggling RTK, Headroom, Caveman, Ponytail	A router dashboard; only the Caveman- and Ponytail-style rules show savings	Those rules installed
17	Caveman: 69 vs 19 tokens	About 8% in agent work, not 65%	Terse block
18	`headroom wrap claude` showing **API Usage Billing**	Real risk	`askai check` fails on `ANTHROPIC_BASE_URL`
19	`/clear` between tasks, the #1 habit	True, and free	`askai ask clear`
20	Disconnect MCP servers (~17,600 tokens each)	Direction right; the figure predates tool deferral	`askai check` counts servers

Popular tip	Verdict
"RTK saves 60–90%"	False at task level: +7% median cost
"Caveman saves 65%"	About 8% on agent tasks
"Cursor Auto is free and unlimited"	Outdated: Auto counts against included usage at low rates
"Delete old transcripts to save tokens"	False: they never enter the context
"Idle Claude Code burns tokens"	False for a stopped session; true for loops, goals and teams left running
"Use the 1M window as insurance, reset around 120K"	True
"Thinking is free"	False: billed as output
"Spend leftover quota before the 5-hour reset"	Only if your weekly cap is not the limit you hit

Sources

Read between 19 and 20 October 2026. Vendor commands and settings change often; check each tool's `/help` after an upgrade.

Official documentation

- Claude Code: costs, environment variables, code intelligence plugins
- Codex: sample configuration, pricing and usage, slash commands, skills
- Antigravity: CLI reference, CLI best practices, rules, skills, MCP, plans, credits
- Cursor: pricing and Auto rates, models and Max Mode

Independent measurements

- JetBrains: RTK tested, Ponytail tested
- Token-saving tools: claims and third-party studies (the author contributes to RTK and says so)
- dotclaude: one heavy week, measured
- Marmelab: which agent plugin to use in 2026

Community, limits and YouTube

- Codex usage limits 2026; Codex vs Claude Code
- Claude Code advisor; advisor vs a bigger model

- Antigravity quotas explained; Cursor usage limits explained; Claude quota and the counter-desync bug
- YouTube: Stop Wasting Claude Tokens (Amine AI); How To Never Run Out Of Codex and Claude Usage Limits (AI LABS); I Tested Usage Limits for Claude, Codex, and Cursor (AI Coding Daily)

Your attachments: the 20-page screenshot PDF and the JPGs, checked page by page above.

AskAI's own evidence: its 7-day measured ledger (the earlier self-built hooks were net negative, so AskAI adds none) and 204 automated tests (202 pass here; the other 2 need real Claude Code transcripts). The tests include a full install followed by uninstall that must leave nothing behind in any of the four tools. The 27-row preview matrix is in `docs/PREVIEW_MATRIX.md`.